Universidad El Bosque · Ingeniería de Sistemas

Bases de Datos 2 · Docente: Christian Felipe Duarte

PL/SQL avanzado en Oracle Database

Cómo funciona PL/SQL por dentro y cómo escribir código que rinda, resista la concurrencia y se pueda diagnosticar

Autor
David Felipe Bejarano Montes

Versión de referencia
Oracle Database 19c (con notas de 21c y 23ai)

Fecha
3 de noviembre de 2026

Contenido

Resumen
Alcance, método y convenciones
1. Arquitectura y ejecución de PL/SQL
2. Diseño modular con paquetes
3. Tipos compuestos y colecciones
4. Procesamiento masivo: BULK COLLECT y FORALL
5. SQL dinámico y prevención de inyección
6. Funciones invocadas desde SQL
7. Transacciones y concurrencia en PL/SQL
8. Disparadores (triggers) avanzados
9. Manejo de errores y diagnóstico
10. Seguridad en PL/SQL
11. Rendimiento, instrumentación y diagnóstico
12. Calidad de código y pruebas
13. Evolución del lenguaje: de 12c a 23ai
14. Relación con los talleres del curso
15. Conclusiones y buenas prácticas
Referencias

Resumen

PL/SQL es el lenguaje de programación de Oracle y se ejecuta dentro del mismo motor de la base de datos. Eso lo hace muy cómodo para trabajar con datos, pero también hace que los errores de diseño se noten más: una sentencia SQL puesta dentro de un bucle, una transacción que dura demasiado o un error que nadie ve pueden afectar a todo el sistema.

En este trabajo reuní lo que hace falta para pasar de escribir procedimientos y triggers que simplemente funcionan a escribir código que aguante volumen, usuarios concurrentes y cambios en el esquema. Los temas son cómo se ejecuta PL/SQL, los paquetes, las colecciones, el procesamiento masivo, el SQL dinámico, las funciones llamadas desde SQL, las transacciones y los bloqueos, los triggers, el manejo de errores, la seguridad, el diagnóstico y las pruebas. En cada uno explico cómo funciona, pongo un ejemplo y comento cuándo conviene usarlo y cuándo no.

> `Ideas principales`
>
> 1. Casi toda la optimización en PL/SQL consiste en cruzar menos veces entre el motor PL/SQL y el motor SQL. Para eso sirven `BULK COLLECT` y `FORALL`, y todavía mejor es resolver todo en una sola sentencia SQL.
> 2. Las variables de enlace son la defensa principal contra la inyección SQL. Lo que no se puede enlazar, como los nombres de tablas, se valida con `DBMS_ASSERT`.
> 3. La consistencia ante concurrencia no viene sola. Hay que elegir entre `FOR UPDATE`, `SKIP LOCKED` o una columna de versión, y bloquear siempre en el mismo orden para evitar interbloqueos.
> 4. Un `WHEN OTHERS` que no relanza el error esconde problemas y deja datos a medias sin que nadie se entere.
> 5. Antes de optimizar hay que medir. El perfilador jerárquico (`DBMS_HPROF`) separa el tiempo de PL/SQL del de SQL y muestra dónde está el cuello de botella.

Alcance, método y convenciones

Alcance y fuentes

Tomé Oracle Database 19c como referencia porque es la versión que usamos en los talleres del curso. Cuando algo es de 21c o de 23ai lo aclaro en el texto. Los datos sobre arquitectura, parámetros de compilación, perfilador jerárquico, novedades de cada versión y SQL dinámico los verifiqué en la documentación oficial de Oracle. En las referencias separé las páginas que consulté directamente de las lecturas de apoyo.

> **Sobre los ejemplos de código**
>
> Los ejemplos están escritos con la sintaxis de la documentación de 19c, pero este documento no trae resultados de ejecución míos. Lo mejor es correrlos en SQL Developer, con un esquema de pruebas, antes de usarlos como evidencia. En la parte de rendimiento (sección 4) dejé un script para medir en lugar de poner números que yo no medí.

Convención de nombres

Todo el código sigue la convención de nombres que usamos en los talleres, para que se vea de un vistazo qué es cada elemento:

Elemento	Prefijo	Ejemplo
Parámetro	`param_`	`param_id_producto`
Procedimiento almacenado	`sp_`	`sp_registrar_salida`
Función	`fn_`	`fn_stock_actual`
Paquete	`pck_`	`pck_inventario`
Trigger	`trg_`	`trg_movimiento_limite`
Cursor	`cur_`	`cur_pendientes`
Excepción	`e_`	`e_stock_insuficiente`
Registro	`r_`	`r_producto`
Índice	`ix_`	`ix_movimiento_producto`
Variable	`v` + letra del tipo	`vv_` (varchar), `vn_` (number), `vd_` (date), `vb_` (boolean), `vc_` (char)

Para las colecciones agregué `ty_` (tipo) y `vt_` (variable de colección), siguiendo la misma idea.

Esquema de laboratorio

Los ejemplos usan un esquema pequeño, separado del de los talleres. Se crea una sola vez:

Código 0.1 · Esquema de laboratorio

```
CREATE TABLE producto (
    id_producto     NUMBER          PRIMARY KEY,
    codigo          VARCHAR2(20)    NOT NULL UNIQUE,
    nombre          VARCHAR2(100)   NOT NULL,
    precio_unitario NUMBER(12,2)    NOT NULL,
    costo_unitario  NUMBER(12,2)    NOT NULL,
    stock           NUMBER(10)      DEFAULT 0 NOT NULL CHECK (stock >= 0),
    activo          CHAR(1)         DEFAULT 'S' NOT NULL
);

CREATE TABLE movimiento_stock (
    id_movimiento NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_producto   NUMBER      NOT NULL REFERENCES producto,
    tipo          VARCHAR2(10) NOT NULL CHECK (tipo IN ('ENTRADA','SALIDA','AJUSTE')),
```

```
    cantidad      NUMBER(10)  NOT NULL,
    fecha         DATE        DEFAULT SYSDATE NOT NULL
);

CREATE TABLE log_errores (
    id_log  NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    fecha   TIMESTAMP DEFAULT SYSTIMESTAMP,
    modulo  VARCHAR2(100),
    codigo  NUMBER,
    mensaje VARCHAR2(4000),
    traza   CLOB
);

CREATE TABLE tarea_cola (
    id_tarea NUMBER PRIMARY KEY,
    estado   VARCHAR2(12) DEFAULT 'PENDIENTE' NOT NULL
);
```

1. Arquitectura y ejecución de PL/SQL

1.1 Dos motores y una frontera

PL/SQL tiene su propio motor para compilar y ejecutar el código. Según la documentación de Oracle, ese motor ejecuta directamente las sentencias procedimentales (asignaciones, condicionales, bucles, llamadas a subprogramas) y le pasa las sentencias SQL al motor SQL de la base de datos [2]. Cada vez que el control pasa de un motor al otro se habla de un *cambio de contexto*, y cada sentencia SQL ejecutada desde PL/SQL implica uno [11].

Un cambio de contexto aislado cuesta muy poco. El problema aparece cuando se repite miles de veces: si un bucle actualiza 10.000 filas de a una, son 10.000 cambios de contexto. Tom Kyte llamaba a esto *slow-by-slow processing* [11]. Es la idea que más conecta el resto del documento, porque casi todas las técnicas de rendimiento de PL/SQL se reducen a cruzar esa frontera menos veces.

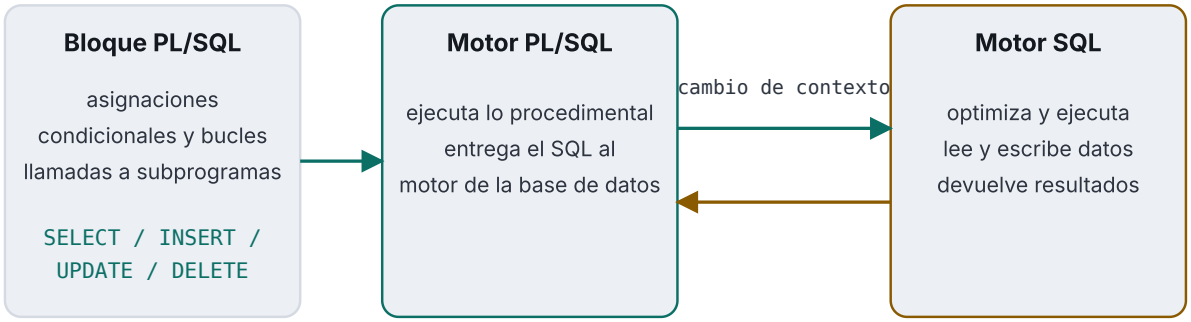


Figura 1.1 · Cada sentencia SQL cruza la frontera entre los dos motores. Elaboración propia a partir de la descripción de [2].

1.2 Unidades PL/SQL

La documentación llama unidad PL/SQL a cualquiera de estas [2]:

- un bloque anónimo;
- una función, un procedimiento, una biblioteca (`LIBRARY`) o un trigger;

- la especificación y el cuerpo de un paquete;
- la especificación y el cuerpo de un tipo.

La diferencia práctica entre un bloque anónimo y una unidad almacenada es que la unidad se guarda en la base de datos con `CREATE [OR REPLACE]` y conserva, junto con sus metadatos, los parámetros con los que se compiló. El bloque anónimo no se guarda, así que no tiene metadatos que reutilizar [2]. Por eso lo almacenado sirve para versionar, dar permisos y reutilizar desde cualquier sesión, y lo anónimo se queda para pruebas y scripts.

1.3 Parámetros de compilación

Cada unidad almacenada se compila con unos parámetros que quedan guardados con ella [2]:

Parámetro	Qué controla
`PLSQL_CODE_TYPE`	`INTERPRETED` (valor predeterminado) o `NATIVE`
`PLSQL_OPTIMIZE_LEVEL`	Nivel de optimización del compilador. Con valor 1 se genera código apto para el depurador
`PLSQL_WARNINGS`	Activa o desactiva advertencias del compilador y define cuáles se tratan como error
`PLSQL_CCFLAGS`	Banderas para la compilación condicional
`PLSCOPE_SETTINGS`	Recolección de datos de identificadores para PL/Scope
`NLS_LENGTH_SEMANTICS`	Si `CHAR` y `VARCHAR2` miden su longitud en bytes o en caracteres

Con `PLSQL_CODE_TYPE = NATIVE` las unidades se compilan a código máquina y se ejecutan sin pasar por el intérprete de bytecode. Con `INTERPRETED` se compilan a bytecode que ejecuta el intérprete de PL/SQL [3]. Hay tres cosas que conviene tener presentes:

- cambiar el parámetro no afecta a las unidades que ya estaban compiladas [3];
- una unidad compilada como `NATIVE` sigue recompilándose como `NATIVE` en sus recompilaciones automáticas [3];
- si `PLSQL_OPTIMIZE_LEVEL` es menor que 2, el compilador genera código interpretado aunque se pida `NATIVE`, y lo avisa con una advertencia [2].

La compilación nativa ayuda más cuando el código hace mucho cálculo (bucles, aritmética, manejo de cadenas) y casi nada cuando el tiempo se va en SQL, porque ahí lo gasta el motor SQL y no el intérprete.

Código 1.1 · Consultar y modificar la configuración de compilación

```
-- ¿Con qué parámetros se compiló cada unidad?
SELECT name, type, plsql_code_type, plsql_optimize_level, plsql_warnings
  FROM user_plsql_object_settings
 WHERE name IN ('PCK_INVENTARIO', 'SP_REGISTRAR_ERROR');

-- Compilar nativamente solo en esta sesión
ALTER SESSION SET PLSQL_OPTIMIZE_LEVEL = 2;
ALTER SESSION SET PLSQL_CODE_TYPE = NATIVE;
ALTER PACKAGE pck_inventario COMPILE BODY;

-- Recompilar conservando los parámetros con que se creó la unidad
ALTER PACKAGE pck_inventario COMPILE REUSE SETTINGS;
```

Para ver esta configuración se usa la vista `ALL_PLSQL_OBJECT_SETTINGS`, o `USER_PLSQL_OBJECT_SETTINGS` si solo interesa el esquema propio [2].

1.4 Dependencias e invalidación

Una unidad almacenada depende de los objetos que usa (tablas, vistas, otros paquetes). Si uno de esos objetos cambia, Oracle marca como `INVALID` las unidades que dependen de él y las vuelve a compilar la siguiente vez que alguien las llama. En un sistema real esto trae tres consecuencias:

- si la unidad invalidada ya no compila, deja de estar disponible hasta que se corrija;
- recompilar con el sistema en uso puede generar esperas, porque la recompilación necesita acceso exclusivo a la unidad;
- separar especificación y cuerpo ayuda: si solo cambia el cuerpo, las unidades que dependen de la especificación no se invalidan.

Código 1.2 · Revisar y reparar objetos inválidos

```
-- Objetos inválidos del esquema
SELECT object_name, object_type, status
  FROM user_objects
 WHERE status = 'INVALID';

-- ¿De qué depende una unidad?
SELECT referenced_owner, referenced_name, referenced_type
  FROM user_dependencies
 WHERE name = 'PCK_INVENTARIO';

-- Errores de compilación de una unidad
SELECT line, position, text
  FROM user_errors
 WHERE name = 'PCK_INVENTARIO'
 ORDER BY sequence;
```

> Para tener en cuenta
>
> Después de cualquier cambio en el esquema conviene revisar `USER_OBJECTS` (estado `INVALID`) y `USER_ERRORS` antes de dar el despliegue por terminado.

2. Diseño modular con paquetes

2.1 Por qué usar paquetes

Un paquete reúne tipos, constantes, excepciones, cursores y subprogramas relacionados en dos partes: la especificación, que es la interfaz pública, y el cuerpo, que es la implementación. Las ventajas que encontré son estas:

- Encapsulación: lo que no aparece en la especificación es privado, así que quien usa el paquete no puede depender de detalles internos.
- Menos invalidaciones: cambiar el cuerpo sin tocar la especificación no invalida a quienes la usan.
- Sobrecarga: varios subprogramas pueden llamarse igual si tienen firmas distintas.

- Estado: las variables del paquete duran lo que dure la sesión, lo que sirve para cachés pequeñas o contadores.
- Carga en bloque: al llamar un subprograma se carga todo el paquete en memoria, y las llamadas siguientes ya no repiten la carga.

Código 2.1 · Especificación y cuerpo de `pck_inventario`

```
CREATE OR REPLACE PACKAGE pck_inventario AS
    gc_stock_maximo CONSTANT NUMBER := 100000;

    -- Excepción pública, asociada al código de aplicación -20101
    e_stock_insuficiente EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_stock_insuficiente, -20101);

    FUNCTION fn_stock_actual (
        param_id_producto IN producto.id_producto%TYPE
    ) RETURN NUMBER;

    PROCEDURE sp_registrar_salida (
        param_id_producto IN producto.id_producto%TYPE,
        param_cantidad    IN NUMBER
    );
END pck_inventario;
/

CREATE OR REPLACE PACKAGE BODY pck_inventario AS
    vn_llamadas NUMBER := 0;   -- estado privado de la sesión

    -- Subprograma privado: no aparece en la especificación
    PROCEDURE sp_validar_cantidad (param_cantidad IN NUMBER) IS
    BEGIN
        IF param_cantidad IS NULL OR param_cantidad <= 0 THEN
            RAISE_APPLICATION_ERROR(-20100, 'La cantidad debe ser positiva');
        END IF;
    END sp_validar_cantidad;

    FUNCTION fn_stock_actual (
        param_id_producto IN producto.id_producto%TYPE
    ) RETURN NUMBER IS
        vn_stock producto.stock%TYPE;
    BEGIN
        SELECT stock INTO vn_stock
          FROM producto
         WHERE id_producto = param_id_producto;
        RETURN vn_stock;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN 0;
    END fn_stock_actual;

    PROCEDURE sp_registrar_salida (
        param_id_producto IN producto.id_producto%TYPE,
        param_cantidad    IN NUMBER
    ) IS
        vn_stock producto.stock%TYPE;
    BEGIN
        sp_validar_cantidad(param_cantidad);

        -- Se bloquea la fila: nadie más puede cambiar el saldo entre leer y escribir
        SELECT stock INTO vn_stock
          FROM producto
         WHERE id_producto = param_id_producto
           FOR UPDATE;

        IF vn_stock < param_cantidad THEN
            RAISE_APPLICATION_ERROR(-20101,
                'Stock insuficiente: hay ' || vn_stock || ' y se piden ' || param_cantidad);
        END IF;
```

```
        UPDATE producto
           SET stock = stock - param_cantidad
         WHERE id_producto = param_id_producto;

        INSERT INTO movimiento_stock (id_producto, tipo, cantidad)
        VALUES (param_id_producto, 'SALIDA', param_cantidad);

        vn_llamadas := vn_llamadas + 1;
    END sp_registrar_salida;

BEGIN
    -- Bloque de inicialización: se ejecuta una vez por sesión,
    -- la primera vez que se referencia el paquete
    vn_llamadas := 0;
END pck_inventario;
/
```

La excepción `e_stock_insuficiente` queda vinculada al código `-20101` con `PRAGMA EXCEPTION_INIT`. Así, quien llama al paquete puede capturarla por su nombre (`WHEN pck_inventario.e_stock_insuficiente`) en lugar de comparar números de error, y dentro del cuerpo se sigue usando `RAISE_APPLICATION_ERROR` para agregar un mensaje que explique qué pasó.

2.2 El estado del paquete

Las variables declaradas en la especificación o en el cuerpo, fuera de los subprogramas, conservan su valor durante la sesión. Es útil para cachés pequeñas, contadores o configuración. Pero el estado es por sesión y no global, y eso importa en dos casos:

- Con agrupaciones de conexiones (*connection pools*), una misma sesión atiende a clientes distintos en momentos distintos. Si el estado asume un solo usuario por sesión, puede pasar información de un cliente a otro.
- Al recompilar el paquete, las sesiones que tenían estado reciben `ORA-04068` («existing state of packages has been discarded») en su siguiente llamada.

Por eso lo razonable es guardar en el estado solo datos que se puedan reconstruir sin pérdida y exponerlos mediante funciones, no mediante variables públicas.

2.3 Limitar quién puede llamar: `ACCESSIBLE BY`

Desde Oracle 12.2 una unidad puede indicar con `ACCESSIBLE BY` qué otras unidades tienen permitido llamarla. Sirve para tener subprogramas internos fuera del paquete, por ejemplo cuando se divide un paquete grande en varios archivos, sin dejarlos disponibles para el resto de la aplicación.

Código 2.2 · Un procedimiento que solo puede invocar `pck_inventario`

```
CREATE OR REPLACE PROCEDURE sp_ajuste_interno (
    param_id_producto IN NUMBER
)
    ACCESSIBLE BY (PACKAGE pck_inventario)
IS
BEGIN
    NULL;  -- lógica interna
END sp_ajuste_interno;
/
```

Si otro bloque intenta llamarlo, falla al compilar con `PLS-00904` (privilegio insuficiente para acceder al objeto).

3. Tipos compuestos y colecciones

3.1 Registros y `%ROWTYPE`

Un registro agrupa campos de tipos distintos bajo un solo nombre. Con `%ROWTYPE` el registro copia la estructura de una tabla, una vista o un cursor, y se actualiza solo cuando la tabla cambia y se recompila el código. `%TYPE` hace lo mismo pero para una sola columna. Los dos evitan un error muy común, que es declarar a mano un `VARCHAR2(50)` cuando la columna ya creció a 100.

3.2 Los tres tipos de colección

PL/SQL tiene tres tipos de colección y cada uno sirve para algo distinto:

Característica	Arreglo asociativo	Tabla anidada	VARRAY
Índice	`PLS_INTEGER` o `VARCHAR2` (clave)	Entero consecutivo desde 1	Entero consecutivo desde 1
Tamaño	Sin límite	Sin límite, crece con `EXTEND`	Tiene un máximo fijo
Densidad	Dispersa por naturaleza	Densa al crearse, puede quedar dispersa al borrar	Siempre densa
Se guarda en una columna	No	Sí (tipo de esquema)	Sí (tipo de esquema)
Operadores de conjunto (`MULTISET`)	No	Sí	No
Uso típico	Cachés en memoria, búsquedas por clave	Lotes, resultados, parámetros de tipo tabla	Listas pequeñas de tamaño acotado

Código 3.1 · Arreglo asociativo: caché de precios por código

```
DECLARE
    TYPE ty_aa_precio IS TABLE OF NUMBER INDEX BY VARCHAR2(20);
    vt_precios ty_aa_precio;
    vv_clave   VARCHAR2(20);
BEGIN
    vt_precios('MARTILLO') := 35000;
    vt_precios('TALADRO')  := 280000;

    -- Recorrido ordenado por clave
    vv_clave := vt_precios.FIRST;
    WHILE vv_clave IS NOT NULL LOOP
        DBMS_OUTPUT.PUT_LINE(vv_clave || ' -> ' || vt_precios(vv_clave));
        vv_clave := vt_precios.NEXT(vv_clave);
    END LOOP;

    -- Acceder a una clave inexistente lanza NO_DATA_FOUND: se verifica antes
    IF NOT vt_precios.EXISTS('SIERRA') THEN
        DBMS_OUTPUT.PUT_LINE('SIERRA no está en la caché');
    END IF;
```

```
END;
/
```

Código 3.2 · Tablas anidadas y operadores de conjunto

```
DECLARE
    TYPE ty_nt_num IS TABLE OF NUMBER;
    vt_a ty_nt_num := ty_nt_num(1, 2, 3, 4);
    vt_b ty_nt_num := ty_nt_num(3, 4, 5);
    vt_c ty_nt_num;
BEGIN
    vt_c := vt_a MULTISET INTERSECT vt_b;          -- (3, 4)
    DBMS_OUTPUT.PUT_LINE('Intersección: ' || vt_c.COUNT);

    vt_c := vt_a MULTISET UNION DISTINCT vt_b;     -- (1, 2, 3, 4, 5)
    DBMS_OUTPUT.PUT_LINE('Unión sin repetidos: ' || vt_c.COUNT);

    vt_c := vt_a MULTISET EXCEPT vt_b;             -- (1, 2)
    DBMS_OUTPUT.PUT_LINE('Diferencia: ' || vt_c.COUNT);
END;
/
```

3.3 Métodos de colección

Las colecciones traen métodos propios: `COUNT`, `FIRST`, `LAST`, `NEXT`, `PRIOR`, `EXISTS`, `EXTEND`, `TRIM` y `DELETE`. Hay dos errores que se repiten bastante. Uno es usar `LAST` o `COUNT` como límite de un bucle sobre una colección dispersa, porque los índices intermedios no existen. El otro es olvidar `EXTEND` antes de asignar a una posición nueva de una tabla anidada, lo que produce `ORA-06533` (subscript beyond count).

3.4 Expresiones calificadas y nuevas formas de iterar

Desde 18c, las expresiones calificadas permiten construir cualquier valor PL/SQL (registros, arreglos asociativos) nombrando los elementos dentro de la misma expresión [8]. La versión 21c agregó controles de iteración ampliados, entre ellos el rango con paso [8].

Código 3.3 · Expresiones calificadas (18c) y rango con paso (21c)

```
DECLARE
    TYPE ty_aa IS TABLE OF NUMBER INDEX BY VARCHAR2(10);
    vt_cupo ty_aa := ty_aa('BRONCE' => 1000, 'PLATA' => 5000, 'ORO' => 20000);  -- 18c
BEGIN
    DBMS_OUTPUT.PUT_LINE('Cupo ORO: ' || vt_cupo('ORO'));

    -- 21c: rango con paso (1, 3, 5, 7, 9)
    FOR vn_i IN 1 .. 10 BY 2 LOOP
        DBMS_OUTPUT.PUT_LINE(vn_i);
    END LOOP;
END;
/
```

> **Compatibilidad**
>
> La primera parte del ejemplo 3.3 necesita Oracle 18c o superior y el bucle con `BY` necesita 21c. En un entorno 19c hay que quitar el bucle.

4. Procesamiento masivo: `BULK COLLECT` y `FORALL`

4.1 El problema de procesar fila por fila

Leer en un bucle y hacer un `UPDATE` por cada fila es el patrón que más problemas de rendimiento causa en PL/SQL. Cada iteración manda una sentencia al motor SQL, con su cambio de contexto [11]. Para resolverlo hay un orden de preferencia:

1. Hacerlo con una sola sentencia SQL (un `UPDATE` o un `MERGE` con la lógica en el `WHERE` y el `SET`). Es lo más rápido y lo más simple.
2. Si la lógica no cabe en SQL, procesar por lotes: `BULK COLLECT` para leer y `FORALL` para escribir.
3. Solo si nada de lo anterior aplica, procesar fila por fila.

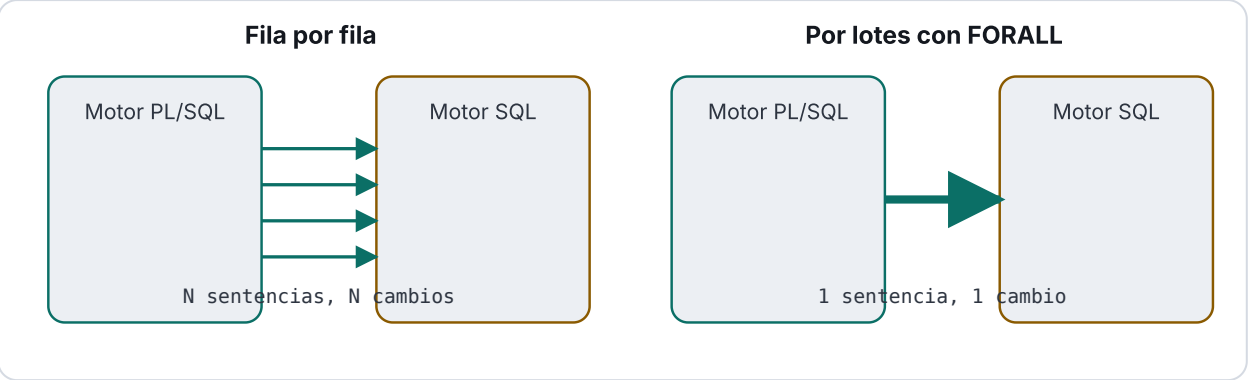


Figura 4.1 · Idea central del procesamiento masivo. Elaboración propia.

Código 4.1 · El antipatrón y su corrección en SQL

```sql
-- Antipatrón: un UPDATE por cada fila
BEGIN
    FOR r_p IN (SELECT id_producto FROM producto WHERE activo = 'S') LOOP
        UPDATE producto
           SET precio_unitario = precio_unitario * 1.05
         WHERE id_producto = r_p.id_producto;
    END LOOP;
    COMMIT;
END;
/

-- Corrección: una sola sentencia, sin bucle
UPDATE producto
   SET precio_unitario = precio_unitario * 1.05
 WHERE activo = 'S';
COMMIT;
```

4.2 `BULK COLLECT` con `LIMIT`

`BULK COLLECT` trae muchas filas en un solo cambio de contexto y las deja en colecciones. Si no se le pone límite, una consulta grande puede agotar la memoria PGA, y por eso se recomienda usar un cursor explícito con `FETCH ... BULK COLLECT INTO ... LIMIT n` dentro de un bucle [11]. El artículo de Oracle dice que rara vez conviene un límite menor a 100 [11]. Además, un bucle `FOR` sobre un cursor ya se optimiza solo para traer 100 filas por vez cuando el nivel de optimización es 2 o mayor, y ese valor no se puede cambiar [11].

> **Ojo con `%NOTFOUND` y `LIMIT`**
>
> En el último `FETCH` con `LIMIT` pueden llegar filas y a la vez activarse `%NOTFOUND`. Si el `EXIT WHEN cur%NOTFOUND` está justo después del `FETCH`, esas últimas filas se pierden [11]. Lo más seguro es salir cuando `colección.COUNT = 0` o mover la condición al final del bucle, después de procesar el lote.

4.3 `FORALL`: escribir por lotes

`FORALL` le manda al motor SQL una sola sentencia de DML (`INSERT`, `UPDATE`, `DELETE` o `MERGE`) junto con todos los valores de la colección. Aunque se escriba parecido, no es un bucle: es una instrucción única que recorre un rango de índices.

Código 4.2 · Lectura con `LIMIT` y escritura con `FORALL`

```
DECLARE
    CURSOR cur_prod IS
        SELECT id_producto, precio_unitario
          FROM producto
         WHERE activo = 'S';

    TYPE ty_nt_id     IS TABLE OF producto.id_producto%TYPE;
    TYPE ty_nt_precio IS TABLE OF producto.precio_unitario%TYPE;
    vt_id     ty_nt_id;
    vt_precio ty_nt_precio;
BEGIN
    OPEN cur_prod;
    LOOP
        FETCH cur_prod BULK COLLECT INTO vt_id, vt_precio LIMIT 500;
        EXIT WHEN vt_id.COUNT = 0;          -- seguro: no pierde el último lote

        FORALL vn_i IN 1 .. vt_id.COUNT
            UPDATE producto
               SET precio_unitario = vt_precio(vn_i) * 1.05
             WHERE id_producto = vt_id(vn_i);
    END LOOP;
    CLOSE cur_prod;
    COMMIT;
END;
/
```

4.4 Errores dentro de un lote: `SAVE EXCEPTIONS`

Por defecto, si una fila del lote falla, `FORALL` se detiene y se deshacen las filas que esa sentencia ya había procesado. Con `SAVE EXCEPTIONS` sigue con el resto y deja el detalle de los fallos en `SQL%BULK_EXCEPTIONS`. Al terminar lanza `ORA-24381` (errores en DML de arreglo), que hay que capturar para revisar qué filas fallaron.

Código 4.3 · Procesar el lote completo y reportar las filas rechazadas

```
DECLARE
    TYPE ty_nt_id IS TABLE OF NUMBER;
    vt_ids ty_nt_id := ty_nt_id(1001, 1002, 1003);
    e_error_lote EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_error_lote, -24381);
BEGIN
    FORALL vn_i IN 1 .. vt_ids.COUNT SAVE EXCEPTIONS
        UPDATE producto
```

```
            SET stock = stock - 10          -- falla si deja el stock negativo (CHECK)
          WHERE id_producto = vt_ids(vn_i);
    COMMIT;
EXCEPTION
    WHEN e_error_lote THEN
        FOR vn_j IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE(
                'Posición ' || SQL%BULK_EXCEPTIONS(vn_j).ERROR_INDEX ||
                ' (id ' || vt_ids(SQL%BULK_EXCEPTIONS(vn_j).ERROR_INDEX) || '): ' ||
                SQLERRM(-SQL%BULK_EXCEPTIONS(vn_j).ERROR_CODE));
        END LOOP;
        COMMIT;   -- confirma lo que sí se pudo aplicar
END;
/
```

4.5 Colecciones dispersas y `RETURNING`

- `FORALL i IN INDICES OF colección` recorre solo los índices que existen en una colección dispersa. `VALUES OF` permite recorrer los índices que indique otra colección.
- `SQL%BULK_ROWCOUNT(i)` dice cuántas filas afectó cada ejecución individual del lote.
- `RETURNING ... BULK COLLECT INTO` recupera de una sola vez los valores de las filas afectadas por un DML.

Código 4.4 · Borrar y recuperar los identificadores eliminados

```
DECLARE
    TYPE ty_nt_id IS TABLE OF movimiento_stock.id_movimiento%TYPE;
    vt_borrados ty_nt_id;
BEGIN
    DELETE FROM movimiento_stock
     WHERE fecha < ADD_MONTHS(SYSDATE, -24)
    RETURNING id_movimiento BULK COLLECT INTO vt_borrados;

    DBMS_OUTPUT.PUT_LINE('Eliminados: ' || vt_borrados.COUNT);
    COMMIT;
END;
/
```

4.6 Cuándo usar cada técnica

Situación	Técnica	Razón
La lógica cabe en SQL	Una sola sentencia (`UPDATE` , `MERGE` , `INSERT ... SELECT`)	No hay cambios de contexto y el optimizador ve todo el trabajo
Hay lógica por fila y mucho volumen	`BULK COLLECT ... LIMIT` + `FORALL`	Se cruza la frontera una vez por lote
Solo se lee, con volumen moderado	Bucle `FOR` sobre cursor	El compilador ya lo optimiza (100 filas por vez, nivel 2 o mayor)
Las filas pueden fallar de forma independiente	`FORALL ... SAVE EXCEPTIONS` o `LOG ERRORS`	Un fallo no detiene al resto
Se necesita el resultado de una modificación	`RETURNING ... BULK COLLECT`	Evita hacer otra consulta después

4.7 Cómo medir la diferencia

En vez de citar cifras de otras fuentes, que dependen del equipo y de la versión, dejo este script para medir la diferencia en el computador donde se ejecute. `DBMS_UTILITY.GET_TIME` devuelve centésimas de segundo.

Código 4.5 · Comparación fila por fila frente a `FORALL`

```
-- Datos de prueba: 100.000 productos
INSERT INTO producto (id_producto, codigo, nombre, precio_unitario, costo_unitario, stock)
SELECT level, 'COD-' || level, 'Producto ' || level, 1000 + level, 500, 10
  FROM dual CONNECT BY level <= 100000;
COMMIT;

DECLARE
    TYPE ty_nt_id IS TABLE OF producto.id_producto%TYPE;
    vt_id ty_nt_id;
    vn_t0 NUMBER;
BEGIN
    SELECT id_producto BULK COLLECT INTO vt_id FROM producto;

    -- 1) Fila por fila
    vn_t0 := DBMS_UTILITY.GET_TIME;
    FOR vn_i IN 1 .. vt_id.COUNT LOOP
        UPDATE producto SET stock = stock + 1 WHERE id_producto = vt_id(vn_i);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Fila por fila: ' || (DBMS_UTILITY.GET_TIME - vn_t0) / 100 || ' s');
    ROLLBACK;

    -- 2) FORALL
    vn_t0 := DBMS_UTILITY.GET_TIME;
    FORALL vn_i IN 1 .. vt_id.COUNT
        UPDATE producto SET stock = stock + 1 WHERE id_producto = vt_id(vn_i);
    DBMS_OUTPUT.PUT_LINE('FORALL:        ' || (DBMS_UTILITY.GET_TIME - vn_t0) / 100 || ' s');
    ROLLBACK;

    -- 3) Una sola sentencia
    vn_t0 := DBMS_UTILITY.GET_TIME;
    UPDATE producto SET stock = stock + 1;
    DBMS_OUTPUT.PUT_LINE('Un UPDATE:     ' || (DBMS_UTILITY.GET_TIME - vn_t0) / 100 || ' s');
    ROLLBACK;
END;
/
```

Conviene correr cada variante varias veces y descartar la primera medición, porque la caché está fría. Lo esperable es que el orden, de menor a mayor tiempo, sea un solo `UPDATE`, luego `FORALL` y al final fila por fila. Cuánto cambia el tiempo entre una y otra es justamente lo que debe anotar quien ejecute la prueba.

5. SQL dinámico y prevención de inyección

5.1 Cuándo hace falta

El SQL dinámico arma y ejecuta sentencias mientras el programa corre. Se necesita cuando el texto completo de la sentencia no se conoce al compilar, por ejemplo en consultas ad hoc, o cuando hay que ejecutar DDL desde PL/SQL, que no lo permite de forma estática [4]. Oracle ofrece dos mecanismos [4]:

- SQL dinámico nativo, con `EXECUTE IMMEDIATE` y `OPEN ... FOR`. Es más fácil de leer y escribir y, según la documentación, más rápido, sobre todo cuando el compilador puede optimizarlo.
- El paquete `DBMS_SQL`, una API para construir, ejecutar y describir sentencias. Es obligatorio cuando no se conoce al compilar el número o el tipo de las columnas o de las variables de enlace, o cuando se quiere devolver resultados de forma implícita sin un parámetro `OUT REF CURSOR`.

Se puede pasar de uno a otro con `DBMS_SQL.TO_REFCURSOR` y `DBMS_SQL.TO_CURSOR_NUMBER` [4]. En la práctica se usa el nativo por defecto y `DBMS_SQL` solo cuando el nativo no alcanza.

5.2 Sintaxis básica

Código 5.1 · `EXECUTE IMMEDIATE` con enlace, `INTO` y `RETURNING`

```
DECLARE
    vv_nombre producto.nombre%TYPE;
    vn_precio producto.precio_unitario%TYPE;
BEGIN
    -- Consulta de una fila con variable de enlace
    EXECUTE IMMEDIATE
        'SELECT nombre FROM producto WHERE id_producto = :id'
        INTO vv_nombre
        USING 1001;

    -- DML con RETURNING INTO
    EXECUTE IMMEDIATE
        'UPDATE producto SET precio_unitario = precio_unitario * 1.1
          WHERE id_producto = :id
          RETURNING precio_unitario INTO :nuevo'
        USING 1001
        RETURNING INTO vn_precio;

    DBMS_OUTPUT.PUT_LINE(vv_nombre || ' ahora cuesta ' || vn_precio);
END;
/
```

Código 5.2 · Consulta dinámica con cursor de referencia

```
DECLARE
    TYPE ty_cur_ref IS REF CURSOR;
    cur_dinamico ty_cur_ref;
    vv_nombre    producto.nombre%TYPE;
BEGIN
    OPEN cur_dinamico FOR
        'SELECT nombre FROM producto WHERE stock < :minimo ORDER BY nombre'
        USING 10;
    LOOP
        FETCH cur_dinamico INTO vv_nombre;
        EXIT WHEN cur_dinamico%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(vv_nombre);
    END LOOP;
    CLOSE cur_dinamico;
END;
/
```

En las sentencias SQL dinámicas, los marcadores de posición (`:id`, `:nuevo`) se asocian con la lista `USING` por posición, no por nombre.

5.3 Inyección SQL

La inyección ocurre cuando datos que escribe el usuario se concatenan en el texto de la sentencia y terminan interpretándose como código. La documentación de Oracle trata el tema en el capítulo de SQL dinámico y propone dos defensas: variables de enlace y validación [4].

Código 5.3 · Vulnerable frente a seguro

```
-- VULNERABLE: el valor se concatena en el texto
CREATE OR REPLACE FUNCTION fn_buscar_inseguro (param_nombre IN VARCHAR2)
    RETURN NUMBER IS
    vn_total NUMBER;
BEGIN
    EXECUTE IMMEDIATE
        'SELECT COUNT(*) FROM producto WHERE nombre = ''' || param_nombre || ''''
        INTO vn_total;
    RETURN vn_total;
END;
/
-- Con param_nombre = ' OR ''1''=''1  la condición se vuelve verdadera para todas las filas.

-- SEGURO: el valor viaja como variable de enlace
CREATE OR REPLACE FUNCTION fn_buscar_seguro (param_nombre IN VARCHAR2)
    RETURN NUMBER IS
    vn_total NUMBER;
BEGIN
    EXECUTE IMMEDIATE
        'SELECT COUNT(*) FROM producto WHERE nombre = :nombre'
        INTO vn_total
        USING param_nombre;
    RETURN vn_total;
END;
/
```

Con variable de enlace el texto de la sentencia siempre es el mismo y el valor nunca se interpreta como SQL. Además el motor reutiliza el plan que ya analizó, lo que reduce el trabajo de análisis (*parse*) en el servidor.

5.4 Lo que no se puede enlazar: `DBMS_ASSERT`

Los identificadores, como nombres de tablas o columnas, no aceptan variables de enlace. Para esos casos se valida la entrada con `DBMS_ASSERT`, que lanza una excepción si el valor no es lo que se esperaba:

Función	Qué hace	Error si falla
`ENQUOTE_LITERAL`	Envuelve en comillas simples y escapa las internas	ORA-06502 si ya tiene comillas sin escapar
`ENQUOTE_NAME`	Envuelve un identificador en comillas dobles (opcionalmente en mayúsculas)	ORA-44003
`SIMPLE_SQL_NAME`	Verifica que sea un nombre simple, sin calificar	ORA-44003
`QUALIFIED_SQL_NAME`	Verifica que sea un nombre que puede ir calificado (`esquema.objeto`)	ORA-44004
`SCHEMA_NAME`	Verifica que el esquema exista	ORA-44001
`SQL_OBJECT_NAME`	Verifica que el objeto exista y sea accesible	ORA-44002

> **Sobre los códigos de error**
>
> Los códigos de la tabla salen de las excepciones documentadas de `DBMS_ASSERT`. Si la aplicación va a depender de ellos, conviene confirmarlos en la documentación de la versión instalada y capturar la excepción en vez de comparar el número.

Código 5.4 · Consulta sobre una tabla elegida dinámicamente

```
CREATE OR REPLACE PROCEDURE sp_contar_filas (
    param_tabla IN  VARCHAR2,
    param_total OUT NUMBER
) IS
    vv_sql VARCHAR2(200);
BEGIN
    -- SQL_OBJECT_NAME rechaza nombres inválidos o inexistentes
    vv_sql := 'SELECT COUNT(*) FROM ' || DBMS_ASSERT.SQL_OBJECT_NAME(param_tabla);
    EXECUTE IMMEDIATE vv_sql INTO param_total;
END sp_contar_filas;
/
```

> **Resumen**
>
> Los valores se enlazan. Los identificadores se validan con `DBMS_ASSERT` o se comparan contra una lista fija escrita en el código. No se concatena texto del usuario sin alguna de las dos protecciones.

6. Funciones invocadas desde SQL

Cuando una sentencia SQL llama a una función PL/SQL, hay un cambio de contexto por cada fila en la que se evalúa. Un `SELECT` sobre un millón de filas que llame a una función serían un millón de cruces. Oracle ofrece varios mecanismos para reducir o evitar ese costo, y en esta sección los comparo.

6.1 `DETERMINISTIC` y `PRAGMA UDF`

`DETERMINISTIC` es una promesa del programador: con los mismos argumentos la función siempre devuelve lo mismo y no depende de datos de tablas ni de la fecha. Le permite al motor reutilizar resultados dentro de una misma sentencia y es necesario para usar la función en un índice basado en funciones. Oracle no comprueba la promesa, así que si es falsa los resultados serán inconsistentes.

`PRAGMA UDF`, disponible desde Oracle 12c, indica que la función se va a usar sobre todo desde SQL y deja que el compilador la genere de modo que llamarla desde una sentencia SQL cueste menos. Solo tiene sentido en funciones que de verdad se llaman desde SQL.

Código 6.1 · Función determinista pensada para SQL

```
CREATE OR REPLACE FUNCTION fn_margen_pct (
    param_precio IN NUMBER,
    param_costo  IN NUMBER
) RETURN NUMBER
    DETERMINISTIC
```

```
IS
    PRAGMA UDF;
BEGIN
    IF param_costo IS NULL OR param_costo = 0 THEN
        RETURN NULL;
    END IF;
    RETURN ROUND((param_precio - param_costo) / param_costo * 100, 2);
END fn_margen_pct;
/

SELECT nombre, fn_margen_pct(precio_unitario, costo_unitario) AS margen_pct
  FROM producto
 ORDER BY margen_pct DESC NULLS LAST;
```

6.2 La cláusula `WITH FUNCTION`

Desde Oracle 12c una consulta puede declarar funciones locales en su cláusula `WITH`. La función existe solo mientras dura la sentencia, no se crea en el esquema y tiene menos sobrecarga que una función almacenada llamada desde SQL. Sirve para lógica auxiliar de un único informe. En algunos contextos (por ejemplo DML o PL/SQL estático) puede hacer falta el hint `WITH_PLSQL`, así que conviene revisar la documentación de la versión que se use.

Código 6.2 · Función local a la consulta

```
WITH
    FUNCTION fn_categoria_precio (param_precio NUMBER) RETURN VARCHAR2 IS
    BEGIN
        RETURN CASE
                   WHEN param_precio >= 200000 THEN 'ALTO'
                   WHEN param_precio >= 50000  THEN 'MEDIO'
                   ELSE 'BAJO'
               END;
    END;
SELECT nombre, precio_unitario, fn_categoria_precio(precio_unitario) AS rango
  FROM producto
/
```

6.3 Caché de resultados: `RESULT_CACHE`

Una función marcada con `RESULT_CACHE` guarda en memoria compartida los resultados para cada combinación de argumentos. Si se la vuelve a llamar con los mismos argumentos, devuelve el valor guardado sin ejecutar el cuerpo. Oracle invalida la entrada automáticamente cuando cambian los datos de las tablas de las que depende la función. En 21c el mecanismo mejoró con control de sentencias y objetos excluidos, historial de funciones y volcado a disco, y el umbral `RESULT_CACHE_EXECUTION_THRESHOLD` decide qué se guarda [8].

Código 6.3 · Caché para consultas de referencia

```
CREATE OR REPLACE FUNCTION fn_nombre_producto (
    param_id_producto IN NUMBER
) RETURN VARCHAR2
    RESULT_CACHE
IS
    vv_nombre producto.nombre%TYPE;
BEGIN
    SELECT nombre INTO vv_nombre
      FROM producto
     WHERE id_producto = param_id_producto;
    RETURN vv_nombre;
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END fn_nombre_producto;
/

-- Inspección y mantenimiento de la caché
SET SERVEROUTPUT ON
EXEC DBMS_RESULT_CACHE.MEMORY_REPORT;
EXEC DBMS_RESULT_CACHE.FLUSH;
```

Conviene	No conviene
Datos de referencia que casi no cambian (catálogos, parámetros)	Tablas que se modifican todo el tiempo, porque la caché se invalida constantemente
Funciones costosas llamadas muchas veces con los mismos argumentos	Funciones que dependen de `SYSDATE`, del usuario o del contexto de la sesión
Pocas combinaciones distintas de argumentos	Argumentos casi únicos en cada llamada, que llenan la memoria sin reutilizarse

6.4 Funciones de tabla en tubería (*pipelined*)

Una función de tabla en tubería devuelve las filas a medida que las produce, con `PIPE ROW`, en lugar de armar toda la colección en memoria. Se consulta con el operador `TABLE` como si fuera una tabla y es útil para transformar datos o encadenar etapas sin guardar resultados intermedios.

Código 6.4 · Función en tubería consultable desde SQL

```
CREATE OR REPLACE TYPE ty_fila_stock AS OBJECT (
    id_producto NUMBER,
    nombre      VARCHAR2(100),
    stock       NUMBER
);
/
CREATE OR REPLACE TYPE ty_tab_stock AS TABLE OF ty_fila_stock;
/
CREATE OR REPLACE FUNCTION fn_stock_bajo (
    param_minimo IN NUMBER
) RETURN ty_tab_stock PIPELINED IS
BEGIN
    FOR r_p IN (SELECT id_producto, nombre, stock
                  FROM producto
                 WHERE stock < param_minimo) LOOP
        PIPE ROW (ty_fila_stock(r_p.id_producto, r_p.nombre, r_p.stock));
    END LOOP;
    RETURN;
END fn_stock_bajo;
/

SELECT * FROM TABLE(fn_stock_bajo(10));
```

6.5 Macros SQL

Una macro SQL es una función que en vez de devolver un valor devuelve un fragmento de texto SQL, que el motor incorpora a la sentencia que la llama. Como la lógica ya es SQL, no hay cambio de contexto al ejecutar. Las macros de tabla, que se usan en la cláusula `FROM` como una vista con

parámetros, se admiten desde la actualización 19.7 [9]. Las macros SQL en general, incluidas las escalares, son una novedad de 21c [8].

Código 6.5 · Macro de tabla (19.7 o superior)

```
CREATE OR REPLACE FUNCTION fn_productos_bajo_minimo (
    param_minimo IN NUMBER
) RETURN VARCHAR2 SQL_MACRO(TABLE) IS
BEGIN
    RETURN 'SELECT id_producto, nombre, stock
              FROM producto
             WHERE stock < param_minimo';
END fn_productos_bajo_minimo;
/

SELECT * FROM fn_productos_bajo_minimo(10);
```

6.6 Comparación

Mecanismo	Costo por fila	Dónde vive	Cuándo usarlo
Función almacenada	Un cambio de contexto	Esquema	Lógica compartida que se llama poco
`PRAGMA UDF`	Menor que la anterior	Esquema	La función se llama casi solo desde SQL
`WITH FUNCTION`	Menor	Solo la sentencia	Lógica auxiliar de un informe
`RESULT_CACHE`	Ninguno cuando acierta la caché	Esquema y memoria compartida	Datos de referencia estables
Macro SQL	Ninguno al ejecutar	Esquema	La lógica se puede expresar como SQL
Expresión SQL directa	Ninguno	La sentencia	Siempre que sea posible, porque es lo más simple

7. Transacciones y concurrencia en PL/SQL

7.1 Atomicidad en tres niveles

Oracle garantiza atomicidad por sentencia: si una sentencia falla, se deshace completa y la transacción sigue. Encima de eso PL/SQL ofrece:

- `COMMIT` y `ROLLBACK` para toda la transacción;
- `SAVEPOINT` y `ROLLBACK TO` para deshacer solo hasta un punto marcado;
- la transacción autónoma, que corre aparte de la transacción que la llama.

Un patrón común en los procedimientos de negocio es marcar un punto al inicio y volver a él si algo falla, dejando que quien llamó decida qué hacer con la transacción exterior:

Código 7.1 · Operación todo o nada con `SAVEPOINT`

```
CREATE OR REPLACE PROCEDURE sp_transferir_stock (
    param_origen  IN producto.id_producto%TYPE,
    param_destino IN producto.id_producto%TYPE,
    param_cant    IN NUMBER
) IS
BEGIN
    SAVEPOINT sp_inicio_transferencia;

    UPDATE producto SET stock = stock - param_cant WHERE id_producto = param_origen;
    UPDATE producto SET stock = stock + param_cant WHERE id_producto = param_destino;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK TO sp_inicio_transferencia;
        RAISE;   -- se informa al llamador; nunca se silencia
END sp_transferir_stock;
/
```

7.2 Transacciones autónomas

`PRAGMA AUTONOMOUS_TRANSACTION` hace que un subprograma corra en una transacción propia. Puede confirmar o revertir sin afectar a la transacción que lo llamó, y al revés. El uso más legítimo es el registro de errores, que tiene que quedar guardado aunque la transacción principal se revierta.

Código 7.2 · Registro de errores que sobrevive al `ROLLBACK`

```
CREATE OR REPLACE PROCEDURE sp_registrar_error (
    param_modulo  IN VARCHAR2,
    param_codigo  IN NUMBER,
    param_mensaje IN VARCHAR2,
    param_traza   IN CLOB DEFAULT NULL
) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO log_errores (modulo, codigo, mensaje, traza)
    VALUES (param_modulo, param_codigo, SUBSTR(param_mensaje, 1, 4000), param_traza);
    COMMIT;   -- obligatorio: una autónoma debe terminar su transacción
END sp_registrar_error;
/
```

> Precauciones
>
> - La transacción autónoma no ve los cambios sin confirmar de la transacción principal.
> - Si termina con cambios pendientes, Oracle lanza `ORA-06519`, así que debe hacer `COMMIT` o `ROLLBACK` antes de retornar.
> - Usarla para saltarse un bloqueo o para forzar un orden de confirmación es señal de mal diseño. Conviene dejarla para auditoría y registro.

7.3 Bloqueo explícito de filas

La lectura consistente de Oracle no bloquea: dos sesiones pueden leer el mismo saldo y escribir cada una su resultado, y así se pierde una actualización. Esto fue justo lo que vimos en el taller de Banco Andino, donde dos retiros sobre la misma cuenta dejaron 800.000 en lugar de 500.000. Cuando una

decisión depende de un valor que otra sesión puede estar cambiando, se bloquea la fila con `SELECT ... FOR UPDATE`, como hace `sp_registrar_salida` en el Código 2.1. Frente a una fila ya bloqueada hay cuatro comportamientos posibles:

Cláusula	Comportamiento	Caso de uso
`FOR UPDATE`	Espera indefinidamente	Transacciones cortas con poca contención
`FOR UPDATE NOWAIT`	Falla de inmediato con `ORA-00054`	Pantallas de usuario que avisan «registro en uso»
`FOR UPDATE WAIT n`	Espera hasta *n* segundos	Reintento con tiempo límite
`FOR UPDATE SKIP LOCKED`	Omite las filas bloqueadas	Colas de trabajo con varios procesadores

Código 7.3 · Procesadores concurrentes de una cola con `SKIP LOCKED`

```
CREATE OR REPLACE PROCEDURE sp_procesar_lote (
    param_tamano IN NUMBER
) IS
    CURSOR cur_pendientes IS
        SELECT id_tarea
          FROM tarea_cola
         WHERE estado = 'PENDIENTE'
         ORDER BY id_tarea
           FOR UPDATE SKIP LOCKED;

    vn_procesadas NUMBER := 0;
BEGIN
    FOR r_t IN cur_pendientes LOOP
        UPDATE tarea_cola
           SET estado = 'PROCESADA'
         WHERE CURRENT OF cur_pendientes;

        vn_procesadas := vn_procesadas + 1;
        EXIT WHEN vn_procesadas >= param_tamano;
    END LOOP;

    COMMIT;   -- después del bucle: confirmar dentro de un cursor FOR UPDATE
              -- libera los bloqueos y provoca ORA-01002 en el siguiente FETCH
END sp_procesar_lote;
/
```

> **Limitar el lote desde PL/SQL**
>
> Si se limita el lote con `ROWNUM` dentro de la misma consulta con `SKIP LOCKED`, puede devolver menos filas de las esperadas, porque `ROWNUM` se evalúa antes de que se descarten las filas bloqueadas. Con un contador en el bucle, como en el código anterior, el reparto entre procesadores es más predecible.

7.4 Interbloqueos y orden de bloqueo

Un interbloqueo (`ORA-00060`) aparece cuando dos transacciones se esperan mutuamente. Oracle detecta el ciclo y aborta una sentencia, no la transacción completa, así que la sesión afectada queda con trabajo a medias y tiene que hacer `ROLLBACK` de forma explícita. Si se asume que Oracle «ya deshizo todo», se puede terminar haciendo `COMMIT` de una transacción incompleta.

La forma de prevenirlo es una regla de diseño: bloquear siempre los recursos compartidos en el mismo orden, sin importar el sentido de la operación de negocio.

Código 7.4 · Orden fijo de bloqueo en una transferencia entre dos filas

```
-- Dentro de sp_transferir_stock: se bloquean ambas filas por id ascendente,
-- sin importar cuál es origen y cuál destino
SELECT id_producto
  FROM producto
 WHERE id_producto IN (param_origen, param_destino)
 ORDER BY id_producto
   FOR UPDATE;

-- Ahora sí, los UPDATE en cualquier orden: ya no puede formarse el ciclo
UPDATE producto SET stock = stock - param_cant WHERE id_producto = param_origen;
UPDATE producto SET stock = stock + param_cant WHERE id_producto = param_destino;
```

7.5 Concurrencia optimista

Cuando los conflictos son raros y hay muchos lectores, bloquear penaliza a todos para evitar un problema que casi no ocurre. La alternativa es una columna de versión: se lee la versión, se modifica con `WHERE version = :leida` y se incrementa. Si `SQL%ROWCOUNT` da 0, alguien más cambió la fila, así que se vuelve a leer y se reintenta. Lo que no se debe hacer es suponer que todo salió bien porque no hubo error: para el motor, un `UPDATE` que afecta cero filas es un resultado válido.

Código 7.5 · Actualización con control de versión

```
-- Dentro de un procedimiento; supone una columna version NUMBER DEFAULT 0 en producto
BEGIN
    UPDATE producto
       SET precio_unitario = param_nuevo_precio,
           version         = version + 1
     WHERE id_producto = param_id_producto
       AND version     = param_version_leida;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20130,
            'El registro cambió desde que fue leído; vuelva a consultarlo');
    END IF;
END;
```

Estrategia	Cuándo usarla	Costo
Pesimista (`FOR UPDATE`)	El choque es poco frecuente pero costoso, por ejemplo una transferencia puntual de alto valor	Las demás sesiones esperan
Optimista (versión)	El recurso se lee mucho y se escribe poco	La aplicación debe reintentar
`SKIP LOCKED`	Reparto de trabajo entre varios procesadores	El orden de procesamiento deja de ser estricto

8. Disparadores (triggers) avanzados

8.1 Puntos de disparo y orden de ejecución

Un trigger de tabla puede ejecutarse en cuatro momentos respecto de una sentencia DML:

1. `BEFORE STATEMENT`: una vez, antes de que la sentencia toque filas.
2. `BEFORE EACH ROW`: por cada fila, antes de modificarla. Aquí se pueden asignar valores a `:NEW`.
3. `AFTER EACH ROW`: por cada fila, después de modificarla.
4. `AFTER STATEMENT`: una vez, cuando la sentencia termina.

Si una tabla tiene varios triggers del mismo tipo, el orden entre ellos no está garantizado a menos que se use la cláusula `FOLLOWS`. Depender de un orden implícito es una causa típica de errores que aparecen y desaparecen.

8.2 El error de tabla mutante

Un trigger de fila no puede consultar ni modificar la tabla sobre la que se está ejecutando la sentencia. Oracle lo impide con `ORA-04091` (tabla mutante), porque en ese momento la tabla está en un estado intermedio. La solución es el trigger compuesto (`COMPOUND TRIGGER`), que junta en una sola unidad los cuatro puntos de disparo y comparte variables entre ellos. Se recolectan los datos fila por fila y se procesan al final, en `AFTER STATEMENT`, cuando la tabla ya no está mutando.

Código 8.1 · Regla de negocio que consulta la propia tabla, sin error de tabla mutante

```
CREATE OR REPLACE TRIGGER trg_movimiento_limite
    FOR INSERT ON movimiento_stock
    COMPOUND TRIGGER

    TYPE ty_nt_id IS TABLE OF NUMBER;
    vt_ids ty_nt_id := ty_nt_id();     -- estado compartido entre los puntos de disparo

    AFTER EACH ROW IS
    BEGIN
        vt_ids.EXTEND;
        vt_ids(vt_ids.COUNT) := :NEW.id_producto;   -- solo se recolecta
    END AFTER EACH ROW;

    AFTER STATEMENT IS
        vn_total NUMBER;
    BEGIN
        FOR vn_i IN 1 .. vt_ids.COUNT LOOP
            SELECT COUNT(*) INTO vn_total
              FROM movimiento_stock
             WHERE id_producto = vt_ids(vn_i)
               AND TRUNC(fecha) = TRUNC(SYSDATE);

            IF vn_total > 100 THEN
                RAISE_APPLICATION_ERROR(-20120,
                    'El producto ' || vt_ids(vn_i) || ' supera 100 movimientos diarios');
            END IF;
        END LOOP;
    END AFTER STATEMENT;

END trg_movimiento_limite;
/
```

8.3 Triggers `INSTEAD OF`

Una vista que combina varias tablas normalmente no admite DML. Un trigger `INSTEAD OF` intercepta la operación sobre la vista y la convierte en modificaciones sobre las tablas base. Así se puede dejar la vista como interfaz de escritura estable aunque el esquema interno cambie.

8.4 Criterios de diseño

Conviene	Conviene evitar
Usar triggers para reglas de integridad que no se pueden declarar como restricción (auditoría, valores derivados)	Poner en un trigger mucha lógica de negocio, porque es invisible para quien lee el procedimiento y difícil de probar
Mantener el trigger corto y delegar el trabajo en un procedimiento de un paquete	Que un trigger dispare otro que modifica la misma tabla, porque las cascadas son difíciles de seguir
Asignar valores en `BEFORE EACH ROW` (`:NEW.columna := ...`)	`COMMIT` dentro de un trigger. La transacción es de quien ejecutó el DML y solo se permite con una transacción autónoma
Recolectar en `AFTER EACH ROW` y procesar en `AFTER STATEMENT`	Consultar la propia tabla desde un trigger de fila
Declarar el orden con `FOLLOWS` cuando importa	Suponer un orden de ejecución entre triggers

Un trigger de fila se ejecuta por cada fila, así que en una carga de un millón de filas sus costos se multiplican. Antes de una carga así hay que evaluar si el trigger se puede deshabilitar un momento (`ALTER TRIGGER ... DISABLE`) y reemplazarlo por una validación equivalente al final.

9. Manejo de errores y diagnóstico

9.1 Cómo funciona una excepción

Cuando ocurre un error en el cuerpo de un bloque, el control salta a la sección `EXCEPTION`. Si no hay un manejador que coincida, la excepción pasa al bloque que contiene al actual y al final llega a quien llamó. Las excepciones pueden ser de tres tipos:

- Predefinidas, como `NO_DATA_FOUND`, `TOO_MANY_ROWS`, `DUP_VAL_ON_INDEX`, `ZERO_DIVIDE` o `VALUE_ERROR`.
- Errores de Oracle sin nombre, a los que se les asigna uno con `PRAGMA EXCEPTION_INIT`, como `ORA-02291` (violación de llave foránea) o `ORA-24381`.
- De aplicación, que se lanzan con `RAISE_APPLICATION_ERROR(código, mensaje)` usando códigos entre -20000 y -20999.

9.2 `WHEN OTHERS`

Un manejador `WHEN OTHERS` captura cualquier error. Tiene sentido en el nivel más externo de una operación, para registrar el error y relanzarlo, y en casi cualquier otro lugar es un defecto. Un `WHEN OTHERS` debería terminar siempre en `RAISE` o en `RAISE_APPLICATION_ERROR`. El compilador ayuda con esto: la advertencia `PLW-06009` avisa cuando un manejador `OTHERS` no relanza la excepción, y desde 21c se puede silenciar de forma explícita con el pragma `SUPPRESSES_WARNING_6009` en una rutina de reporte propia [8].

Código 9.1 · Silenciar un error frente a registrarlo y propagarlo

```
-- MAL: el llamador cree que todo salió bien
BEGIN
    pck_inventario.sp_registrar_salida(1001, 5);
EXCEPTION
    WHEN OTHERS THEN
        NULL;
END;
/

-- BIEN: se registra, se conserva la causa y se propaga
BEGIN
    pck_inventario.sp_registrar_salida(1001, 5);
EXCEPTION
    WHEN pck_inventario.e_stock_insuficiente THEN
        RAISE;   -- error de negocio esperado: el llamador decide qué hacer
    WHEN OTHERS THEN
        sp_registrar_error(
            param_modulo  => 'pck_inventario.sp_registrar_salida',
            param_codigo  => SQLCODE,
            param_mensaje => SQLERRM,
            param_traza   => DBMS_UTILITY.FORMAT_ERROR_BACKTRACE);
        RAISE;
END;
/
```

9.3 Dónde ocurrió el error

`SQLERRM` dice qué falló pero no dónde. Hay dos herramientas para eso:

- `DBMS_UTILITY.FORMAT_ERROR_BACKTRACE` devuelve en texto la pila de llamadas, con la unidad y la línea donde se originó el error.
- El paquete `UTL_CALL_STACK` (desde 12c) entrega esa misma información de forma estructurada: profundidad del error, número y mensaje de cada nivel, y unidad y línea de cada paso.

Código 9.2 · Traza estructurada con `UTL_CALL_STACK`

```
EXCEPTION
    WHEN OTHERS THEN
        FOR vn_i IN 1 .. UTL_CALL_STACK.ERROR_DEPTH LOOP
            DBMS_OUTPUT.PUT_LINE(
                'ORA-' || UTL_CALL_STACK.ERROR_NUMBER(vn_i) || ': ' ||
                UTL_CALL_STACK.ERROR_MSG(vn_i));
        END LOOP;
        FOR vn_i IN 1 .. UTL_CALL_STACK.BACKTRACE_DEPTH LOOP
            DBMS_OUTPUT.PUT_LINE(
                UTL_CALL_STACK.BACKTRACE_UNIT(vn_i) || ' línea ' ||
                UTL_CALL_STACK.BACKTRACE_LINE(vn_i));
        END LOOP;
        RAISE;
```

> **Capturar los valores al inicio del manejador**
>
> `SQLCODE`, `SQLERRM` y la traza corresponden a la excepción en curso. Si el manejador ejecuta antes otras sentencias que también producen errores, esos valores pueden cambiar. Lo más seguro es copiarlos a variables locales en la primera línea del manejador, o pasarlos como argumentos, como en el Código 9.1.

9.4 Errores por fila en DML masivo: `LOG ERRORS`

En las cargas masivas donde algunas filas pueden ser inválidas, el DML puede guardar cada fila rechazada en una tabla de errores y seguir con las demás, sin necesidad de PL/SQL. La tabla de errores se crea con `DBMS_ERRLOG.CREATE_ERROR_LOG`.

Código 9.3 · Carga que no se detiene por filas inválidas

```
BEGIN
    DBMS_ERRLOG.CREATE_ERROR_LOG(dml_table_name => 'PRODUCTO');   -- crea ERR$_PRODUCTO
END;
/

INSERT INTO producto (id_producto, codigo, nombre, precio_unitario, costo_unitario)
SELECT id_origen, codigo_origen, nombre_origen, precio_origen, costo_origen
  FROM carga_temporal
LOG ERRORS INTO err$_producto ('carga-2026-11') REJECT LIMIT UNLIMITED;

-- Filas rechazadas, con la causa de cada una
SELECT ora_err_number$, ora_err_mesg$, codigo
  FROM err$_producto
 WHERE ora_err_tag$ = 'carga-2026-11';
```

Comparado con `FORALL ... SAVE EXCEPTIONS`, `LOG ERRORS` también funciona en sentencias SQL puras y deja los errores guardados en una tabla que se puede consultar. `SAVE EXCEPTIONS` es mejor cuando la reacción al error tiene que ejecutarse dentro del propio programa.

10. Seguridad en PL/SQL

10.1 Derechos del definidor y del invocador

Por defecto, un subprograma se ejecuta con los privilegios de su dueño (derechos del definidor). Esto sirve como herramienta de seguridad: se puede dar `EXECUTE` sobre un paquete sin dar acceso directo a las tablas que maneja, y la aplicación solo puede hacer lo que el paquete permite. Con `AUTHID CURRENT_USER` el subprograma usa los privilegios de quien lo llama (derechos del invocador), lo que va bien para utilidades genéricas que deben trabajar con los datos de cada usuario.

	Derechos del definidor (`AUTHID DEFINER`)	Derechos del invocador (`AUTHID CURRENT_USER`)
Privilegios usados	Los del dueño del código	Los de quien lo llama
Roles	No se aplican dentro de la unidad	Se aplican los roles habilitados del invocador
Resolución de nombres	En el esquema del dueño	En el esquema del invocador
Uso típico	Aplicaciones: exponer operaciones y no tablas	Utilidades genéricas sobre objetos del propio usuario
Riesgo principal	Un dueño con demasiados privilegios puede permitir una escalada si hay inyección	La resolución de nombres se puede manipular si no se califican los objetos

Código 10.1 · Exponer operaciones y no tablas

```
-- El dueño del esquema (APP_OWNER) crea el paquete con derechos del definidor
CREATE OR REPLACE PACKAGE pck_inventario AUTHID DEFINER AS
    -- ...
END pck_inventario;
/

-- La cuenta de la aplicación solo recibe EXECUTE, nunca acceso a las tablas
GRANT EXECUTE ON pck_inventario TO app_usuario;
REVOKE ALL ON producto FROM app_usuario;
```

En derechos del invocador, desde 12c una unidad solo puede usar los privilegios de quien la llama si este le concedió `INHERIT PRIVILEGES` a su dueño (por defecto, `PUBLIC` lo tiene concedido). Revocarlo para dueños no confiables reduce el riesgo de que código ajeno aproveche los privilegios de quien lo llama.

10.2 Capas de defensa

1. Mínimo privilegio: cada cuenta recibe solo `EXECUTE` sobre los paquetes que necesita.
2. Nada de SQL dinámico concatenado: variables de enlace y `DBMS_ASSERT` (sección 5).
3. Restringir quién llama con `ACCESSIBLE BY` (sección 2.3) en los subprogramas internos.
4. Calificar los nombres de objeto (`esquema.objeto`) en el código con derechos del invocador.
5. Usar `wrap` solo si se distribuye código. Oculta el texto fuente, pero no es un cifrado fuerte ni reemplaza a los privilegios.
6. No dejar secretos en el código. Contraseñas, claves y tokens van fuera del texto PL/SQL. La documentación de Oracle tiene una sección sobre cómo proteger información sensible en PL/SQL.

11. Rendimiento, instrumentación y diagnóstico

Antes de optimizar hay que medir. Una mejora sobre la parte equivocada no reduce el tiempo total. Oracle tiene herramientas distintas según lo que se quiera averiguar.

11.1 Qué herramienta usar para cada pregunta

Pregunta	Herramienta
¿Dónde se va el tiempo? ¿Es PL/SQL o SQL?	Perfilador jerárquico `DBMS_HPROF` [6] [7]
¿En qué orden se ejecutaron los subprogramas?	API de traza `DBMS_TRACE` [7]
¿Dónde se declara y se usa cada identificador? ¿Hay variables sin usar?	PL/Scope (`PLSCOPE_SETTINGS`)
¿Qué cubren las pruebas?	`DBMS_PLSQL_CODE_COVERAGE`
¿Qué módulo de la aplicación está ejecutando esta sesión?	`DBMS_APPLICATION_INFO`
¿Por qué tarda tanto la sentencia SQL?	Plan de ejecución (`EXPLAIN PLAN` , `DBMS_XPLAN`), que vimos en el taller de Banco Andino

11.2 El perfilador jerárquico

El perfilador jerárquico informa dónde gasta el tiempo un programa PL/SQL, organizado por las llamadas entre subprogramas. A diferencia de un perfilador plano, muestra quién llama a quién, cuántas veces y cuánto tarda cada ruta, incluyendo a los descendientes. Registra por separado el tiempo de SQL y el de PL/SQL, y no hace falta preparar ni recompilar el código [6].

Tiene dos componentes [6]:

- La recolección de datos, integrada en la máquina virtual de PL/SQL y controlada con `START_PROFILING` y `STOP_PROFILING`.
- El analizador (`DBMS_HPROF.ANALYZE`), que procesa el archivo en bruto y llena las tablas de análisis.

Las métricas, en microsegundos [6]:

Métrica	Columna	Significado
Llamadas	`CALLS`	Cuántas veces se llamó al subprograma
Tiempo de función	`FUNCTION_ELAPSED_TIME`	Tiempo en el propio subprograma, sin descendientes
Tiempo de subárbol	`SUBTREE_ELAPSED_TIME`	Tiempo del subprograma más el de sus descendientes
Tiempo de descendientes	(derivado)	Subárbol menos función

Código 11.1 · Perfilar una ejecución y consultar el resultado

```
-- 1) Una sola vez, como administrador: directorio de salida y permisos
CREATE DIRECTORY plshprof_dir AS '/ruta/segura/plshprof';
GRANT READ, WRITE ON DIRECTORY plshprof_dir TO app_owner;
GRANT EXECUTE ON DBMS_HPROF TO app_owner;

-- 2) Como el usuario: tablas de análisis (una vez)
EXEC DBMS_HPROF.CREATE_TABLES;

-- 3) Perfilar
BEGIN
    DBMS_HPROF.START_PROFILING('PLSHPROF_DIR', 'inventario.trc');
END;
/
BEGIN
    pck_inventario.sp_registrar_salida(1001, 5);   -- el código a medir
END;
/
BEGIN
    DBMS_HPROF.STOP_PROFILING;
END;
/

-- 4) Analizar el archivo
DECLARE
    vn_runid NUMBER;
BEGIN
    vn_runid := DBMS_HPROF.ANALYZE(location => 'PLSHPROF_DIR',
                                   filename => 'inventario.trc');
    DBMS_OUTPUT.PUT_LINE('runid = ' || vn_runid);
END;
/

-- 5) Las funciones más costosas de la corrida
SELECT module, function, calls, function_elapsed_time, subtree_elapsed_time
  FROM dbmshp_function_info
```

```
 WHERE runid = :runid
 ORDER BY function_elapsed_time DESC;
```

Las tablas que crea el analizador son `DBMSHP_RUNS` (una fila por análisis), `DBMSHP_FUNCTION_INFO` (una por subprograma y corrida) y `DBMSHP_PARENT_CHILD_INFO` (una por cada pareja llamador y llamado) [6]. También se puede generar un informe HTML directamente desde la línea de comandos con la utilidad `plshprof`.

11.3 PL/Scope y trazas

PL/Scope no mide tiempo. Es una facilidad del compilador que registra los identificadores y las sentencias del código y los expone en vistas como `ALL_IDENTIFIERS` y `ALL_STATEMENTS`. Sirve para análisis estático: saber dónde se usa una variable, qué impacto tendría renombrarla o qué código nunca se referencia.

Código 11.2 · Variables declaradas y nunca referenciadas

```
ALTER SESSION SET PLSCOPE_SETTINGS = 'IDENTIFIERS:ALL';
ALTER PACKAGE pck_inventario COMPILE BODY;

SELECT d.name, d.line
  FROM user_identifiers d
 WHERE d.object_name = 'PCK_INVENTARIO'
   AND d.type        = 'VARIABLE'
   AND d.usage       = 'DECLARATION'
   AND NOT EXISTS (SELECT 1
                     FROM user_identifiers r
                    WHERE r.signature = d.signature
                      AND r.usage     = 'REFERENCE');
```

La API de traza `DBMS_TRACE` registra el orden en que se ejecutan los subprogramas que se indiquen, con el nivel de detalle que se elija, en tablas que se pueden consultar [7]. Responde a «¿qué se ejecutó y en qué orden?», no a «¿qué fue lento?».

11.4 Instrumentación mínima para producción

Un programa que no dice qué está haciendo es muy difícil de diagnosticar cuando falla en producción. `DBMS_APPLICATION_INFO` deja el módulo y la acción visibles en `V$SESSION` y en las trazas SQL.

Código 11.3 · Marcar el módulo y la acción de la sesión

```
BEGIN
    DBMS_APPLICATION_INFO.SET_MODULE(
        module_name => 'pck_inventario',
        action_name => 'sp_registrar_salida');
    -- ... trabajo ...
    DBMS_APPLICATION_INFO.SET_ACTION('actualizando stock');
    -- ... trabajo ...
    DBMS_APPLICATION_INFO.SET_MODULE(NULL, NULL);   -- limpiar al terminar
END;
/

-- Desde otra sesión: qué está haciendo cada cliente
SELECT sid, module, action
  FROM v$session
 WHERE module = 'pck_inventario';
```

11.5 Advertencias del compilador y compilación condicional

Con `PLSQL_WARNINGS` el compilador avisa de problemas antes de ejecutar el código: variables sin usar, código inalcanzable, conversiones implícitas costosas, manejadores `OTHERS` que no relanzan. La compilación condicional permite incluir código de diagnóstico solo cuando una bandera lo pide, sin costo en producción.

Código 11.4 · Advertencias y código de depuración condicional

```
ALTER SESSION SET PLSQL_WARNINGS = 'ENABLE:ALL';
ALTER SESSION SET PLSQL_CCFLAGS  = 'depuracion:TRUE';

CREATE OR REPLACE PROCEDURE sp_ejemplo_condicional IS
BEGIN
    $IF $$depuracion $THEN
        DBMS_OUTPUT.PUT_LINE('Entrando a ' || $$PLSQL_UNIT);
    $END
    NULL;   -- lógica real
END sp_ejemplo_condicional;
/

-- Ver las advertencias del compilador
SELECT line, attribute, text
  FROM user_errors
 WHERE name = 'SP_EJEMPLO_CONDICIONAL'
   AND attribute = 'WARNING';
```

Con `PLSQL_OPTIMIZE_LEVEL = 2` (el valor predeterminado) el compilador reordena y simplifica el código. Con 3 activa además la expansión agresiva de subprogramas en línea (*inlining*), que también se puede pedir a mano con `PRAGMA INLINE(subprograma, 'YES')`. Si se necesita depurar con un depurador, se compila con nivel 1 [2].

12. Calidad de código y pruebas

12.1 Pruebas unitarias con utPLSQL

El código que vive en la base de datos también necesita pruebas automáticas. utPLSQL es un framework de pruebas unitarias de código abierto para PL/SQL. Las pruebas son paquetes con anotaciones en comentarios que el framework encuentra y ejecuta.

Código 12.1 · Un paquete de pruebas

```
CREATE OR REPLACE PACKAGE test_inventario AS
    --%suite(Pruebas de pck_inventario)

    --%test(Devuelve 0 cuando el producto no existe)
    PROCEDURE stock_producto_inexistente;

    --%test(Rechaza una cantidad negativa)
    --%throws(-20100)
    PROCEDURE rechaza_cantidad_negativa;
END test_inventario;
/

CREATE OR REPLACE PACKAGE BODY test_inventario AS
    PROCEDURE stock_producto_inexistente IS
    BEGIN
        ut.expect(pck_inventario.fn_stock_actual(-1)).to_equal(0);
```

```
    END;

    PROCEDURE rechaza_cantidad_negativa IS
    BEGIN
        pck_inventario.sp_registrar_salida(1001, -5);
    END;
END test_inventario;
/

-- Ejecutar
EXEC ut.run('test_inventario');
```

12.2 Lista de revisión de código

Área	Pregunta de revisión
Rendimiento	¿Hay SQL dentro de un bucle que podría ser una sola sentencia o un `FORALL`?
Memoria	¿Todo `BULK COLLECT` tiene `LIMIT` o un tamaño acotado por naturaleza?
Seguridad	¿Hay SQL dinámico? ¿Los valores van enlazados y los identificadores validados?
Errores	¿Algún `WHEN OTHERS` termina sin `RAISE`?
Concurrencia	¿Se lee un valor y luego se escribe sin bloquear ni usar versión?
Bloqueos	¿Se toman varios bloqueos siempre en el mismo orden?
Transacciones	¿Quién decide el `COMMIT`? ¿Algún subprograma reutilizable confirma por su cuenta?
Triggers	¿Tienen lógica que debería estar en un paquete? ¿Consultan su propia tabla en un trigger de fila?
Nombres	¿Se respetan los prefijos y el nombre dice qué hace el elemento?
Pruebas	¿Hay una prueba para el caso normal y una para cada excepción de negocio?

12.3 Control de versiones y despliegue

El código PL/SQL se versiona como cualquier otro: un archivo por unidad en un repositorio, escrito con `CREATE OR REPLACE` para que el script de despliegue se pueda repetir sin problema. Los cambios de estructura (tablas) se manejan como migraciones numeradas, separadas del código de las unidades. Después de cada despliegue se revisa que no queden objetos inválidos (Código 1.2).

13. Evolución del lenguaje: de 12c a 23ai

PL/SQL siguió cambiando después de 19c. La tabla resume lo que agregaron las versiones siguientes y de dónde sale cada dato.

Versión	Novedad	Para qué sirve
12c	`WITH FUNCTION`, `PRAGMA UDF`, `UTL_CALL_STACK`, `INHERIT PRIVILEGES`	Menos sobrecarga al llamar PL/SQL desde SQL, mejor diagnóstico y más control de seguridad
12.2	`ACCESSIBLE BY`	Controlar qué unidades pueden llamar a otra

Versión	Novedad	Para qué sirve
18c	Expresiones calificadas para cualquier valor PL/SQL [8]	Construir registros y colecciones en una sola expresión
19.7	Macros SQL de tabla [9]	Vistas con parámetros sin cambio de contexto
21c	Macros SQL, iteradores extendidos, tipo `JSON`, caché de resultados mejorada, `SUPPRESSES_WARNING_6009` [8]	Más expresividad y mejor rendimiento
23ai	Ver el detalle abajo [10]	Menos código repetitivo en SQL y DDL

23ai: novedades de SQL que cambian el trabajo diario

Según el tutorial oficial de novedades de SQL de Oracle Database 23ai [10]:

- `SELECT` sin `FROM`: `SELECT CURRENT_DATE;` ya no necesita `DUAL`.
- Tipo `BOOLEAN` nativo en columnas, que acepta valores como `1`, `0`, `YES`, `NO`, `TRUE` y `FALSE`.
- `IF [NOT] EXISTS` en DDL, que hace repetibles los `CREATE` y `DROP` sin bloques PL/SQL de verificación.
- `INSERT` de varias filas y constructor de tabla (`VALUES` en línea).
- Alias en `GROUP BY`, `RETURNING` con valores antiguos y nuevos en `UPDATE` y `MERGE`, y uniones en `UPDATE`.
- Anotaciones y dominios SQL, que son metadatos y tipos reutilizables a nivel de esquema.

Código 13.1 · Ejemplos de 23ai (no ejecutables en 19c)

```
DROP TABLE IF EXISTS producto_demo;
CREATE TABLE IF NOT EXISTS producto_demo (
    id     NUMBER PRIMARY KEY,
    activo BOOLEAN
);
INSERT INTO producto_demo (id, activo) VALUES (1, 'YES'), (2, 0);   -- BOOLEAN y multifila
SELECT CURRENT_DATE;                                                -- sin FROM
```

> **Portabilidad**
>
> Los talleres del curso se ejecutan en Oracle 19c, así que el código que se entrega debe evitar estas novedades. Las incluyo para ubicar hacia dónde va el lenguaje.

14. Relación con los talleres del curso

Varios de los conceptos de esta investigación aparecen, con otro nombre, en los dos talleres que están publicados en el portafolio. La tabla relaciona cada pieza del trabajo con la sección que explica el criterio de fondo.

Pieza del taller	Concepto de la investigación	Sección
Taller PL/SQL: `SP_AJUSTAR_STOCK` con `SELECT ... FOR UPDATE`	Bloqueo pesimista para evitar la pérdida de actualización	7.3
Taller PL/SQL: `BULK COLLECT` y `FORALL` (Parte 3)	Procesamiento por lotes y menos cambios de contexto	4
Taller PL/SQL: cursor con `WHERE CURRENT OF`	Bloqueo de filas durante un recorrido	7.3
Taller PL/SQL: `SYS_REFCURSOR`	Cursores de referencia y SQL dinámico	5
Taller PL/SQL: `SP_REGISTRAR_ERROR`	Transacción autónoma para el registro de errores	7.2 y 9
Taller PL/SQL: `TRG_DETALLE_TOTAL` (`COMPOUND TRIGGER`)	Evitar la tabla mutante	8.2
Taller PL/SQL: `PKG_VENTAS` con `SAVEPOINT`	Paquetes y atomicidad parcial	2 y 7.1
Banco Andino: actualización perdida y `FOR UPDATE`	Pesimista frente a optimista	7.3 y 7.5
Banco Andino: `SKIP LOCKED`	Reparto de colas entre procesadores	7.3
Banco Andino: interbloqueo `ORA-00060`	Orden fijo de bloqueo	7.4
Banco Andino: índices por función y planes de ejecución	Revisar el SQL antes de tocar el PL/SQL	11.1

15. Conclusiones y buenas prácticas

Lo difícil de PL/SQL no es escribir código que funcione, sino que siga funcionando cuando crece el volumen, cuando llegan sesiones concurrentes, cuando el esquema cambia y cuando alguien intenta abusar del sistema. De todo lo revisado, me quedo con cuatro ideas:

1. Cruzar la frontera entre los dos motores tiene un costo, y cada técnica de rendimiento busca cruzarla menos veces (SQL puro, lotes, funciones en tubería, macros).
2. El motor solo protege lo que se le pide proteger. La consistencia ante concurrencia se diseña con bloqueos, versiones y un orden fijo.
3. Un error silenciado es peor que uno visible. Hay que registrarlo, relanzarlo y dejar la traza de dónde ocurrió.
4. Lo que no se mide ni se prueba no se puede garantizar, así que conviene perfilar antes de optimizar y automatizar las pruebas.

Lista de buenas prácticas

1. Si la lógica cabe en una sola sentencia SQL, escribir esa sentencia.
2. Usar `LIMIT` en todo `BULK COLLECT` y evaluar `COUNT = 0` en lugar de `%NOTFOUND` justo después del `FETCH`.
3. Enlazar siempre los valores y validar los identificadores con `DBMS_ASSERT`.
4. Agrupar la lógica en paquetes, con la especificación más pequeña posible.
5. Hacer que un `WHEN OTHERS` registre y relance, nunca que silencie.
6. Bloquear los recursos compartidos siempre en el mismo orden.

7. Preferir `SKIP LOCKED` para repartir trabajo y el control de versión donde los conflictos son raros.
8. Dejar las transacciones autónomas para el registro y la auditoría.
9. Mantener los triggers cortos y usar `COMPOUND TRIGGER` si se necesita consultar la propia tabla.
10. Dar a las cuentas de aplicación solo `EXECUTE` sobre los paquetes.
11. Medir con `DBMS_HPROF` antes de optimizar y marcar módulo y acción con `DBMS_APPLICATION_INFO`.
12. Compilar con las advertencias activadas y revisar `USER_ERRORS` y los objetos inválidos después de cada despliegue.
13. Escribir pruebas para el caso normal y para cada excepción de negocio.

Referencias

Fuentes consultadas directamente

1. Oracle. *Oracle Database PL/SQL Language Reference, 19c: Table of Contents.* docs.oracle.com/.../19/lnpls/toc.htm. Se usó para ubicar los capítulos de referencia.
2. Oracle. *Architecture of PL/SQL* (PL/SQL Language Reference, edición 23). docs.oracle.com/.../23/lnpls/architecture-pl-sql.html. Motor PL/SQL, unidades, parámetros de compilación y compilación nativa.
3. Oracle. *PLSQL_CODE_TYPE* (Database Reference, 19c). docs.oracle.com/.../19/refrn/PLSQL_CODE_TYPE.html.
4. Oracle. *PL/SQL Dynamic SQL* (PL/SQL Language Reference, 19c). docs.oracle.com/.../19/lnpls/dynamic-sql.html. Se consultó la introducción del capítulo (mecanismos y cuándo usar cada uno).
5. Oracle. *Oracle Database PL/SQL Language Reference, 19c*, capítulos de colecciones, SQL estático, subprogramas, paquetes, triggers, manejo de errores, optimización y compilación condicional. Ver la tabla de contenido [1]. Se usaron como lectura de apoyo.
6. Oracle. *Using the PL/SQL Hierarchical Profiler* (Database Development Guide, 19c). docs.oracle.com/.../19/adfns/hierarchical-profiler.html.
7. Oracle. *Profiling and Tracing PL/SQL Programs* (PL/SQL Language Reference, edición 23). docs.oracle.com/.../23/lnpls/profiling-and-tracing-pl-sql-programs.html.
8. Oracle. *Changes in This Release for PL/SQL Language Reference* (21c). docs.oracle.com/.../21/lnpls/release-changes.html.
9. Oracle. *Release Update 19.7* (Database New Features, 19c). docs.oracle.com/.../19/newft/ru-19-07.html.
10. Oracle. *Oracle Database 23ai SQL Features* (tutorial). docs.oracle.com/en/learn/db23ai-sql-features.
11. Feuerstein, S. *Solving the row-by-row problem.* Oracle Magazine / Oracle Connect Blog. blogs.oracle.com/connect/post/solving-the-row-by-row-problem.

Aclaración sobre las fuentes

Las secciones 2, 3, 6 (la parte de `RESULT_CACHE`, `DETERMINISTIC` y `WITH FUNCTION`), 7, 8, 9 y 10 se apoyan en conocimiento general del lenguaje y en la estructura de los capítulos de la documentación [5], sin citar una página puntual para cada afirmación. Si algún dato se va a presentar como cita textual (por ejemplo los números de error de `DBMS_ASSERT` o la sintaxis exacta de una cláusula), conviene contrastarlo con el capítulo correspondiente de la documentación 19c.